

MEAN Stack Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

1. Programming & Web Development Fundamentals

- Introduction to software development
- What is full-stack development?
- What is the MEAN Stack?
- MEAN Stack architecture
- Frontend vs backend development
- Client-server architecture
- Three-tier architecture
- Single Page Applications
- How web applications work
- Browser and server responsibilities
- HTTP and HTTPS
- Request-response lifecycle
- REST APIs
- JSON
- Development, testing, staging, and production environments
- IDE setup
- Visual Studio Code
- Browser developer tools
- Terminal and command-line fundamentals
- Introduction to Git and GitHub
- Introduction to Node.js ecosystem
- Introduction to npm

Part I — Web Fundamentals

2. HTML5

- Introduction to HTML
- HTML document structure
- Elements
- Tags
- Attributes
- Headings
- Paragraphs
- Text formatting
- Hyperlinks
- Images
- Lists
- Tables
- Forms
- Input elements
- Buttons
- Select controls
- Radio buttons
- Checkboxes
- File input

- Semantic HTML
- Header
- Footer
- Nav
- Main
- Section
- Article
- Aside
- Audio
- Video
- iframe
- Meta tags
- Favicons
- HTML entities
- Data attributes
- Accessibility fundamentals
- SEO-friendly HTML

3. CSS3

- Introduction to CSS
- CSS syntax
- Inline CSS
- Internal CSS
- External CSS
- CSS selectors
- Element selector
- Class selector
- ID selector
- Attribute selectors
- Combinators
- Colors
- Backgrounds
- Typography
- Web fonts
- Units
- px
- %
- em
- rem
- vh
- vw

CSS Box Model

- Content
- Padding
- Border
- Margin
- Box sizing

Layout & Positioning

- Display
- Position
- Relative

- Absolute
- Fixed
- Sticky
- Float concepts
- Z-index
- Overflow

4. Modern CSS Layouts

Flexbox

- Flex container
- Flex items
- Flex direction
- Justify content
- Align items
- Flex wrap
- Gap
- Flexible layouts

CSS Grid

- Grid container
- Rows
- Columns
- Grid areas
- Gap
- Responsive grids

Responsive Web Design

- Mobile-first development
- Media queries
- Responsive units
- Responsive images
- Breakpoints
- Adaptive layouts

5. Advanced CSS

- Pseudo-classes
- Pseudo-elements
- CSS variables
- Transitions
- Transformations
- Animations
- Keyframes
- Shadows
- Gradients
- Responsive typography
- CSS architecture basics
- Reusable utility classes

6. Bootstrap / Tailwind CSS

Students may learn one or both.

Bootstrap

- Containers
- Grid system
- Breakpoints
- Buttons
- Forms
- Tables
- Cards
- Alerts
- Navigation bars
- Dropdowns
- Modals
- Offcanvas
- Responsive utilities

Tailwind CSS

- Utility-first CSS
- Layout utilities
- Flexbox
- Grid
- Spacing
- Typography
- Responsive classes
- State variants
- Reusable component patterns
- Customization

Part II — JavaScript Programming

7. JavaScript Fundamentals

- Introduction to JavaScript
- JavaScript engine
- Adding JavaScript to HTML
- Statements
- Comments
- Variables
- var
- let
- const
- Identifiers
- Data types
- Primitive types
- Reference types
- typeof
- Type conversion
- Type coercion

Operators

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators

- Conditional operator
- Nullish coalescing operator

8. JavaScript Control Flow

Conditions

- if
- if-else
- else-if
- Nested conditions
- switch

Loops

- for
- while
- do-while
- for...of
- for...in
- Nested loops
- break
- continue

9. JavaScript Functions

- Function declaration
- Function expression
- Parameters
- Arguments
- Return values
- Default parameters
- Anonymous functions
- Arrow functions
- Callback functions
- Higher-order functions
- Rest parameters
- Spread operator
- Recursion
- Closures
- Lexical scope
- Function scope
- Block scope
- Hoisting
- Immediately Invoked Function Expressions concepts

10. Arrays & Objects

Arrays

- Creating arrays
- Accessing elements
- Modifying arrays
- Array methods
- push
- pop
- shift

- unshift
- slice
- splice
- concat
- includes
- indexOf

Functional Array Methods

- map
- filter
- reduce
- find
- findIndex
- some
- every
- sort
- forEach

Objects

- Creating objects
- Properties
- Methods
- Nested objects
- Object iteration
- Object destructuring
- Object spread
- Object methods
- Optional chaining

11. Strings, Dates & Built-In Objects

- Strings
- String methods
- Template literals
- Regular expressions
- Math object
- Date object
- JSON
- `JSON.stringify()`
- `JSON.parse()`
- Set
- Map
- WeakSet concepts
- WeakMap concepts

12. Object-Oriented JavaScript

- Objects and prototypes
- Prototype chain
- Constructor functions
- Classes
- Constructors
- Properties
- Methods

- Encapsulation
- Inheritance
- Polymorphism
- Static members
- Private fields
- Getters
- Setters
- `this`
- `call`
- `apply`
- `bind`

13. Modern JavaScript — ES6+

- `let` and `const`
- Arrow functions
- Template literals
- Destructuring
- Object destructuring
- Array destructuring
- Spread operator
- Rest operator
- Default parameters
- Modules
- `import`
- `export`
- Classes
- Promises
- `Async/await`
- Optional chaining
- Nullish coalescing
- Enhanced object literals

Part III — Browser & DOM Programming

14. DOM Manipulation

- Document Object Model
- DOM tree
- Selecting elements
- `getElementById`
- `querySelector`
- `querySelectorAll`
- Changing content
- Changing attributes
- Changing styles
- Creating elements
- Removing elements
- Traversing DOM
- Classes and `classList`

15. Events

- Event-driven programming
- Click events
- Keyboard events
- Mouse events
- Form events
- Input events
- Event listeners
- Event object
- Event bubbling
- Event capturing
- Event delegation
- Preventing default behavior

16. Forms & Validation

- Form controls
- Reading form data
- Client-side validation
- Required fields
- Pattern validation
- Custom validation
- Error messages
- Form submission
- Preventing invalid submissions

17. Browser Storage

- Cookies
- Local Storage
- Session Storage
- Storage APIs
- JSON storage
- Authentication storage considerations
- Security concerns

Part IV — Asynchronous JavaScript

18. Asynchronous Programming

- Synchronous vs asynchronous execution
- JavaScript runtime
- Call stack
- Web APIs
- Callback queue
- Event loop
- Callbacks
- Callback hell
- Promises
- Promise states
- Promise chaining
- Error handling
- `async`
- `await`

- try/catch with async code
- Parallel async operations
- Promise.all
- Promise.allSettled

19. AJAX & API Communication

- AJAX fundamentals
- HTTP methods
- Fetch API
- GET
- POST
- PUT
- PATCH
- DELETE
- Request headers
- Request bodies
- Response objects
- JSON
- API error handling
- Loading states
- Working with public APIs

Part V — TypeScript

Angular development requires a strong TypeScript foundation.

20. TypeScript Fundamentals

- JavaScript vs TypeScript
- Installing TypeScript
- TypeScript compiler
- Configuration
- tsconfig.json
- Static typing
- Type inference
- Primitive types
- Arrays
- Tuples
- Enums
- any
- unknown
- never
- void
- Union types
- Intersection types
- Literal types

21. TypeScript Functions & Objects

- Typed parameters
- Return types
- Optional parameters
- Default parameters

- Function types
- Object types
- Type aliases
- Interfaces
- Readonly properties
- Optional properties

22. TypeScript OOP

- Classes
- Constructors
- Access modifiers
- Public
- Private
- Protected
- Properties
- Methods
- Inheritance
- Abstract classes
- Interfaces
- Static members
- Generics

23. Advanced TypeScript

- Generics
- Generic constraints
- Utility types
- Partial
- Pick
- Omit
- Record
- keyof
- typeof
- Type guards
- Discriminated unions
- Decorators concepts
- Modules
- Namespace concepts

Part VI — Angular Frontend Development

24. Angular Fundamentals

- Introduction to Angular
- Angular ecosystem
- Angular architecture
- SPA architecture
- Angular CLI
- Project creation
- Project structure
- Angular application lifecycle
- Components

- Templates
- Component metadata
- Standalone components
- Application bootstrap

25. Angular Components

- Creating components
- Component structure
- Template
- Styles
- Selectors
- Nested components
- Parent-child components
- Reusable components

26. Angular Data Binding

- Interpolation
- Property binding
- Event binding
- Two-way binding
- ngModel
- Component communication
- Input properties
- Output properties
- EventEmitter

27. Angular Directives

- Built-in directives
- Structural directives
- Attribute directives
- Conditional rendering
- List rendering
- ngClass
- ngStyle
- Custom directives
- Modern Angular control-flow syntax

28. Angular Pipes

- Built-in pipes
- Date pipe
- Currency pipe
- Decimal pipe
- Percent pipe
- Uppercase/lowercase
- JSON pipe
- Async pipe
- Custom pipes
- Pure vs impure pipes concepts

29. Angular Services & Dependency Injection

- Services
- Why services are required
- Creating services
- Injectable
- Dependency injection
- Providers
- Service scope
- Singleton services
- Separating business logic from UI

30. Angular Routing

- Angular Router
- Route configuration
- Router outlet
- Navigation
- Route parameters
- Query parameters
- Child routes
- Nested routes
- Lazy loading
- Route guards
- Authentication guards
- 404 pages
- Redirects

31. Angular Forms

Template-Driven Forms

- Form controls
- ngModel
- Validation
- Error handling

Reactive Forms

- FormControl
- FormGroup
- FormBuilder
- Validators
- Nested forms
- FormArray
- Dynamic forms
- Custom validators
- Async validators

32. Angular HTTP Client

- HttpClient
- GET requests
- POST
- PUT
- PATCH
- DELETE
- Request headers

- Query parameters
- Error handling
- API services
- Observable responses
- Environment-based API URLs

33. Angular Interceptors

- HTTP interceptors
- Adding authorization headers
- JWT tokens
- Centralized error handling
- Request logging
- Loading indicators
- Refresh-token concepts

34. RxJS & Reactive Programming

- Introduction to RxJS
- Observables
- Observer
- Subscription
- Subjects
- BehaviorSubject
- ReplaySubject concepts
- Operators
- map
- filter
- tap
- switchMap
- mergeMap
- concatMap
- debounceTime
- distinctUntilChanged
- catchError
- Unsubscribing
- Async pipe
- Memory leak prevention

35. Angular State Management

- Local component state
- Shared service state
- BehaviorSubject-based state
- Signals
- Computed state
- Effects
- Introduction to NgRx
- Store
- Actions
- Reducers
- Selectors
- Effects concepts

36. Angular UI Development

- Reusable components
- Layout components
- Navigation
- Dashboards
- Tables
- Forms
- Modals
- Alerts
- Notifications
- Pagination
- Search
- Filtering
- Sorting
- Loading indicators
- Skeleton loaders
- Responsive Angular applications

Optional libraries:

- Angular Material
- Bootstrap
- Tailwind CSS

Part VII — Node.js Backend Development

37. Node.js Fundamentals

- Introduction to Node.js
- Why Node.js?
- Browser JavaScript vs Node.js
- Node.js architecture
- V8 engine
- Event-driven architecture
- Non-blocking I/O
- Single-threaded event loop concepts
- Installing Node.js
- Node REPL
- Creating Node.js applications

38. Node.js Modules

- CommonJS
- ES Modules
- `require`
- `module.exports`
- `import`
- `export`
- Built-in modules
- Creating custom modules
- Third-party modules

39. npm & Package Management

- npm
- npm registry
- package.json
- Installing packages
- Local dependencies
- Dev dependencies
- Semantic versioning
- Package lock
- npm scripts
- Managing dependencies
- Security audits

40. Node.js Core Modules

- fs
- path
- http
- https
- url
- events
- os
- crypto
- Streams
- Buffers

41. Node.js Event Loop

- Event loop
- Call stack
- Callback queue
- Microtasks
- Timers
- Asynchronous execution
- Non-blocking I/O
- EventEmitter
- Performance considerations

42. File Handling in Node.js

- Reading files
- Writing files
- Updating files
- Deleting files
- Directories
- Synchronous vs asynchronous file APIs
- Streams
- Buffers
- File uploads concepts

Part VIII — Express.js Backend Development

43. Express.js Fundamentals

- Introduction to Express.js

- Installing Express
- Creating an Express server
- Application object
- Request object
- Response object
- Routing
- Route parameters
- Query parameters
- Request body
- JSON middleware

44. Express Routing

- GET routes
- POST routes
- PUT routes
- PATCH routes
- DELETE routes
- Router
- Modular routing
- Route parameters
- Query strings
- Nested routes
- RESTful routing conventions

45. Express Middleware

- What is middleware?
- Application-level middleware
- Router-level middleware
- Built-in middleware
- Third-party middleware
- Custom middleware
- Error-handling middleware
- Authentication middleware
- Authorization middleware
- Logging middleware

46. REST API Development

- REST architecture
- RESTful resources
- HTTP methods
- HTTP status codes
- Request headers
- Request bodies
- Response structures
- CRUD APIs
- API validation
- Pagination
- Filtering
- Sorting
- Searching
- API versioning

- Error responses
- API documentation

47. API Architecture & Project Structure

Recommended project structure:

Routes → Controllers → Services → Models → Database

Topics:

- Separation of concerns
- Controller layer
- Service layer
- Repository concepts
- Data-access layer
- Utility functions
- Middleware
- Configuration
- Error classes
- Validation schemas
- Reusable API responses

Part IX — MongoDB

48. MongoDB Fundamentals

- Introduction to databases
- SQL vs NoSQL
- What is MongoDB?
- Document-oriented databases
- Database
- Collection
- Document
- BSON
- MongoDB installation
- MongoDB Compass
- MongoDB Atlas

49. MongoDB CRUD

- Creating databases
- Creating collections
- Insert one
- Insert many
- Find
- Find one
- Update one
- Update many
- Delete one
- Delete many

50. MongoDB Queries

- Query operators

- Comparison operators
- Logical operators
- Array operators
- Projection
- Sorting
- Limiting
- Skipping
- Pagination
- Regular-expression searches

51. MongoDB Data Modeling

- Embedded documents
- Referenced documents
- One-to-one relationships
- One-to-many relationships
- Many-to-many relationships
- Schema design
- Denormalization
- Designing for query patterns

52. MongoDB Aggregation Framework

- Aggregation pipeline
- \$match
- \$project
- \$group
- \$sort
- \$limit
- \$skip
- \$unwind
- \$lookup
- \$count
- Aggregation expressions
- Reporting queries

53. MongoDB Indexing & Performance

- Indexes
- Single-field indexes
- Compound indexes
- Unique indexes
- Text indexes
- Query optimization
- Explain plans
- Index selection
- Performance considerations

Part X — Mongoose

54. Mongoose Fundamentals

- What is Mongoose?
- ODM concepts

- Connecting Node.js to MongoDB
- Schema
- Models
- Documents
- Data types
- Schema validation
- Default values
- Required fields

55. Mongoose CRUD

- Create
- Find
- FindById
- Update
- Delete
- Query chaining
- Filtering
- Sorting
- Pagination

56. Mongoose Relationships

- References
- ObjectId
- Population
- Nested schemas
- Embedded documents
- One-to-many relationships
- Many-to-many modeling
- Virtuals

57. Advanced Mongoose

- Schema methods
- Static methods
- Middleware/hooks
- Pre hooks
- Post hooks
- Validation
- Custom validators
- Transactions
- Aggregation
- Indexes
- Query performance

Part XI — Authentication & Security

58. User Authentication

- Authentication vs authorization
- User registration
- Login
- Logout

- Password hashing
- bcrypt
- Password validation
- User profiles
- Forgot password concepts
- Reset password workflow
- Email verification concepts

59. JWT Authentication

- JWT structure
- Header
- Payload
- Signature
- Access token
- Refresh token
- Token generation
- Token verification
- Token expiration
- Authentication middleware
- Angular JWT integration
- Protected routes

60. Authorization

- Role-Based Access Control
- Roles
- Permissions
- Admin users
- Standard users
- Resource ownership
- Route protection
- Angular guards
- Backend authorization middleware

61. OAuth & Social Authentication

- OAuth 2.0 concepts
- OpenID Connect concepts
- Google authentication concepts
- Microsoft authentication concepts
- Social login
- Single Sign-On introduction

62. Web Application Security

- Password hashing
- HTTPS
- TLS concepts
- CORS
- CSRF
- XSS
- NoSQL injection
- Command injection
- Input validation

- Output encoding
- Secure HTTP headers
- Helmet
- Rate limiting
- Brute-force protection
- Cookie security
- JWT security
- Secrets management
- Environment variables
- OWASP Top 10 awareness

Part XII — MEAN Stack Integration

63. Complete MEAN Architecture

Students should understand:

Angular → Express.js → Node.js → MongoDB

Detailed flow:

Angular Component → Angular Service → REST API → Express Route → Controller → Service → Mongoose → MongoDB

64. Angular + Node.js API Integration

- Angular HttpClient
- Creating API services
- Connecting Angular with Express
- Environment configuration
- CORS
- GET
- POST
- PUT
- PATCH
- DELETE
- Authentication
- Authorization
- Error handling
- Loading indicators
- Form submissions
- File uploads
- Pagination
- Filtering
- Sorting
- Searching

65. Full-Stack Authentication

Complete workflow:

Angular Login → Express API → MongoDB User Validation → JWT → Angular Storage → Route Guard

Topics:

- Registration

- Login
- JWT
- Refresh-token concepts
- HTTP interceptor
- Auth guard
- User roles
- Protected backend endpoints
- Role-based frontend navigation

Part XIII — API Development Best Practices

66. REST API Design

- Resource-oriented API design
- URI naming
- HTTP verbs
- HTTP status codes
- CRUD conventions
- JSON response structure
- Validation
- Error handling
- Pagination
- Filtering
- Sorting
- Searching
- API versioning
- Idempotency
- Rate limiting

67. API Validation

Common options:

- Joi
- Zod
- express-validator

Topics:

- Request-body validation
- Query validation
- Parameter validation
- Data sanitization
- Custom validation
- Centralized validation errors

68. API Documentation

- OpenAPI specification
- Swagger
- Swagger UI
- Documenting endpoints
- Request schemas
- Response schemas
- Authentication documentation

- Testing APIs using Swagger

Part XIV — File Handling

69. File Upload & Storage

- Multipart forms
- Multer
- File upload API
- Upload validation
- File size restrictions
- File type validation
- Image upload
- Profile pictures
- Product images
- Local storage vs cloud storage
- Object-storage concepts

Part XV — Real-Time Development

70. WebSockets & Socket.IO

- HTTP vs WebSockets
- Real-time communication
- Socket.IO
- Server setup
- Client setup
- Events
- Rooms
- Broadcasting
- User presence
- Real-time notifications
- Chat applications
- Angular integration

Part XVI — Advanced Node.js

71. Node.js Performance

- Event loop
- Async I/O
- Streams
- Buffers
- CPU-bound vs I/O-bound tasks
- Worker threads concepts
- Clustering concepts
- Performance monitoring
- Memory management
- Avoiding event-loop blocking

72. Background Jobs

- Background processing
- Queues
- Job processors
- Email jobs
- Scheduled tasks
- Cron jobs
- BullMQ concepts
- Redis-backed queues
- Retry strategies
- Failed jobs

73. Redis & Caching

- What is caching?
- Redis fundamentals
- Key-value storage
- Application caching
- Session storage
- API response caching
- Cache expiration
- Cache invalidation
- Distributed caching concepts
- Rate limiting

74. Email & Notification Services

- Sending emails
- Nodemailer concepts
- Registration confirmation
- Forgot-password emails
- Notifications
- HTML email templates
- Third-party email services concepts

Part XVII — Testing

75. JavaScript & Node.js Testing

- Testing fundamentals
- Unit testing
- Integration testing
- End-to-end testing
- Test-driven development concepts

Possible frameworks:

- Jest
- Vitest concepts
- Mocha concepts

Topics:

- Test suites
- Test cases
- Assertions
- Mocking

- Spies
- Fixtures
- Testing asynchronous code

76. Express API Testing

- API unit tests
- Integration testing
- Supertest
- Testing routes
- Testing controllers
- Testing services
- Authentication tests
- Database test strategies

77. Angular Testing

- Angular testing fundamentals
- Component testing
- Service testing
- Dependency mocking
- HTTP testing
- Router testing
- Forms testing
- Unit testing

78. API Testing Tools

- Postman
- Collections
- Environments
- Variables
- Authorization
- Automated test scripts
- Swagger testing
- API workflow testing

Part XVIII — Git & GitHub

79. Git Fundamentals

- Version control
- Git installation
- Repository creation
- `git init`
- `git clone`
- `git status`
- `git add`
- `git commit`
- `git push`
- `git pull`
- `git fetch`

80. Branching & Collaboration

- Branches
- Feature branches
- Merging
- Rebase basics
- Merge conflicts
- Pull requests
- Code reviews
- GitHub Issues
- GitHub Projects
- Tags
- Releases
- .gitignore
- README
- Branching strategies

Part XIX — Software Engineering Practices

81. Clean Code & Development Standards

- Naming conventions
- Modular code
- DRY
- KISS
- YAGNI
- Separation of concerns
- Reusable components
- Reusable services
- Environment configuration
- Error handling
- Logging
- Documentation
- Code reviews

82. SOLID Principles

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Applied using TypeScript and Node.js.

83. Design Patterns

Introduction to:

- Module Pattern
- Factory Pattern
- Singleton Pattern
- Repository Pattern
- Service Pattern
- Strategy Pattern
- Observer Pattern
- Dependency Injection

- MVC pattern
- Middleware pattern

84. Agile Development

- Agile
- Scrum
- Product backlog
- Sprint
- User stories
- Acceptance criteria
- Sprint planning
- Daily stand-up
- Code review
- Jira fundamentals
- Team collaboration

Part XX — Deployment & DevOps

85. Linux Fundamentals

- Linux commands
- Directory structure
- Files
- Permissions
- Processes
- Package management
- Environment variables
- Networking basics
- SSH
- Logs
- Running Node.js applications

86. Deployment Environments

Students should understand:

Local Development → Testing → Staging → Production

Topics:

- Environment-specific configuration
- Secrets management
- Logging
- Monitoring
- Build generation
- Database connection configuration
- Production best practices

87. Production Node.js Deployment

- Running Node applications in production
- Process managers
- PM2 concepts
- Reverse proxy

- Nginx concepts
- HTTPS
- Domain setup
- SSL certificates
- Environment variables
- Application logs
- Restart strategies

88. Docker

- Introduction to containers
- Containers vs virtual machines
- Docker images
- Docker containers
- Dockerfile
- Ports
- Volumes
- Networks
- Environment variables
- .dockerignore

Students containerize:

- Angular frontend
- Node/Express backend
- MongoDB

Final architecture:

Angular Container → Node.js/Express Container → MongoDB Container

89. Docker Compose

- Multi-container applications
- Docker Compose file
- Services
- Networks
- Volumes
- Environment configuration
- Running complete MEAN Stack applications

90. CI/CD

- Continuous Integration
- Continuous Delivery
- Continuous Deployment
- Build pipelines
- Automated testing
- Environment deployment
- Deployment pipelines

Possible tools:

- GitHub Actions
- Azure DevOps concepts
- GitLab CI concepts

Typical pipeline:

Commit → Install → Build → Test → Docker Image → Deploy

Part XXI — Cloud Deployment

91. Cloud Fundamentals

- Cloud computing
- IaaS
- PaaS
- SaaS
- Regions
- Availability
- Virtual machines
- App hosting
- Managed databases
- Object storage
- Domains
- DNS
- SSL/TLS

92. Deploying MEAN Applications

Possible deployment platforms:

- AWS
- Microsoft Azure
- Google Cloud
- Render
- Railway
- Vercel / Netlify for frontend
- MongoDB Atlas

Students should learn to deploy:

- Angular frontend
- Node.js API
- MongoDB database
- Environment variables
- Domain
- HTTPS

Part XXII — Application Monitoring

93. Logging

- Application logs
- Error logs
- Access logs
- Log levels
- Structured logging
- Winston concepts
- Morgan concepts
- Request logging
- Correlation IDs

94. Monitoring & Observability Fundamentals

- Application monitoring
- Metrics
- Logs
- Traces
- Performance monitoring
- Error tracking
- Health checks
- Availability monitoring
- Production troubleshooting

Part XXIII — Microservices Fundamentals

95. Monolith vs Microservices

- Monolithic architecture
- Modular monolith
- Microservices
- Advantages and disadvantages
- Service decomposition
- Independent deployments
- Database-per-service concept
- Communication patterns

96. Node.js Microservices

- Creating independent Node.js services
- REST communication
- Service-to-service calls
- API Gateway concepts
- Authentication between services
- Message queues
- Event-driven communication
- Redis
- RabbitMQ concepts
- Kafka concepts
- Fault tolerance concepts
- Retry patterns
- Circuit breaker concepts

Part XXIV — AI-Assisted MEAN Stack Development

97. AI-Assisted Software Development

Modern developers should know how to work effectively with coding assistants.

Tools can include:

- GitHub Copilot
- ChatGPT
- Claude
- Cursor
- AI coding agents

Topics:

- Prompting for code
- Generating Angular components
- Generating Express APIs
- MongoDB query generation
- TypeScript code generation
- Debugging with AI
- Refactoring
- Unit-test generation
- API test generation
- Documentation generation
- Code review using AI
- Understanding legacy code
- Generating regular expressions
- Generating validation schemas
- AI-assisted database design
- AI-assisted Git workflow
- Context engineering
- Working with coding agents
- Validating AI-generated code
- Security risks of generated code
- Avoiding blind copy-paste development

Part XXV — Generative AI Integration

98. Generative AI Fundamentals for MEAN Developers

- Introduction to Generative AI
- Large Language Models
- Prompt engineering
- System prompts
- User prompts
- LLM APIs
- Tokens
- Context windows
- Structured output
- JSON responses

99. Integrating LLMs with Node.js

- Calling LLM APIs from Node.js
- Server-side AI integration
- Protecting API keys
- Prompt templates
- Chat history
- Streaming responses
- Error handling
- Rate limiting
- Structured responses
- Tool/function calling concepts

Example architecture:

Angular → Node.js/Express → LLM API

100. RAG Fundamentals

- What is RAG?
- Embeddings
- Vector representations
- Vector databases
- Chunking
- Retrieval
- Semantic search
- Prompt augmentation
- Grounded responses

Example architecture:

Angular → Express → LLM + Vector Database → Knowledge Base

101. AI-Powered MEAN Applications

Students can build:

- AI chatbot
- AI customer-support application
- Document Q&A system
- Resume analyzer
- AI content generator
- AI search application
- AI-powered learning portal
- AI-based FAQ assistant
- AI product-description generator

Part XXVI — Data Structures & Algorithms

102. DSA with JavaScript

Data Structures

- Arrays
- Strings
- Stack
- Queue
- Linked list
- Hash maps
- Sets
- Trees
- Binary trees
- Binary search trees
- Heap
- Graphs

Algorithms

- Linear search
- Binary search
- Bubble sort
- Selection sort
- Insertion sort
- Merge sort

- Quick sort
- Recursion
- Two-pointer technique
- Sliding window
- Hashing
- Basic dynamic programming

Complexity

- Time complexity
- Space complexity
- Big-O notation

Part XXVII — Projects

103. JavaScript Mini Projects

Examples:

- Calculator
- Todo application
- Quiz application
- Expense tracker
- Weather application
- Notes application
- Form validation system

104. Angular Mini Projects

- Admin dashboard
- Task management UI
- E-commerce frontend
- Employee management UI
- Student portal
- API-based application
- Portfolio application

105. Node.js / Express API Projects

- Student Management API
- Employee Management API
- Product Management API
- Authentication API
- Blog API
- Task Management API
- E-commerce backend

106. MongoDB Projects

- Student database
- Product catalog
- Employee management database
- Blogging database
- E-commerce database
- Social media data model

107. Full-Stack Project 1 — Task Management System

Technology:

Angular + Node.js + Express.js + MongoDB

Features:

- User registration
- Login
- JWT authentication
- User dashboard
- Task creation
- Task editing
- Categories
- Priorities
- Status tracking
- Search
- Filtering
- Pagination
- Role-based access

108. Full-Stack Project 2 — E-Commerce Application

Features:

- Registration
- Authentication
- Product catalog
- Categories
- Product search
- Filters
- Product details
- Shopping cart
- Wishlist
- Orders
- Product images
- Admin panel
- Role management
- Payment-gateway concepts
- REST API

109. Full-Stack Project 3 — Job Portal

Features:

- Candidate registration
- Employer registration
- Login
- JWT authentication
- Job posting
- Job search
- Job application
- Resume upload
- Candidate dashboard
- Employer dashboard
- Role-based authorization

- Notifications

Part XXVIII — Capstone Project

110. Industry-Style Capstone Project

Students should complete the entire software development lifecycle:

Requirements → UI Design → Database Design → Angular Frontend → Node/Express Backend → MongoDB → REST APIs → Authentication → Testing → Docker → CI/CD → Cloud Deployment

Possible capstone projects:

- Learning Management System
- CRM
- Hospital Management System
- Recruitment Portal
- E-commerce platform
- Food-delivery application
- Project management system
- Hotel-booking system
- Social networking application
- Inventory management platform
- AI-powered customer-support system
- AI-enabled Learning Management System

Part XXIX — GitHub & Portfolio Development

111. GitHub Profile Building

- Creating a professional GitHub profile
- GitHub profile README
- Repository organization
- Repository naming
- Meaningful commits
- Professional README files
- Project screenshots
- Architecture diagrams
- API documentation
- Setup instructions
- Demo links
- Deployment links
- GitHub contribution history
- Open-source fundamentals

Part XXX — Career & Placement Preparation

112. Resume Preparation

- MEAN Stack developer resume
- Full-stack developer resume
- ATS-friendly resume
- Technical skills presentation

- Project descriptions
- Highlighting achievements
- GitHub links
- Portfolio links

113. LinkedIn Profile Building

- Professional headline
- About section
- Skills
- Projects
- Certifications
- GitHub integration
- Portfolio presentation
- Networking fundamentals

114. JavaScript Interview Preparation

- Variables
- Scope
- Hoisting
- Closures
- `this`
- Prototypes
- Event loop
- Promises
- Async/await
- Arrays
- Objects
- ES6+
- Memory management concepts
- Coding questions

115. TypeScript Interview Preparation

- Interfaces
- Types
- Classes
- Generics
- Union types
- Utility types
- Type guards
- Type inference
- Access modifiers

116. Angular Interview Preparation

- Components
- Templates
- Binding
- Directives
- Pipes
- Services
- Dependency injection
- Routing

- Guards
- Forms
- HttpClient
- Interceptors
- RxJS
- Observables
- Subjects
- Signals
- Component lifecycle
- Change detection concepts

117. Node.js Interview Preparation

- Node.js architecture
- Event loop
- Non-blocking I/O
- Async programming
- Modules
- npm
- Streams
- Buffers
- Events
- Error handling
- Scalability
- Performance

118. Express.js Interview Preparation

- Routing
- Middleware
- Controllers
- REST APIs
- Error handling
- Authentication
- Authorization
- CORS
- API security
- Project structure

119. MongoDB Interview Preparation

- SQL vs NoSQL
- Collections
- Documents
- CRUD
- Aggregation
- Indexing
- Data modeling
- Embedding vs referencing
- Transactions
- Query optimization
- Mongoose

120. Web & API Interview Preparation

- HTTP
- HTTPS
- REST
- HTTP methods
- Status codes
- Cookies
- Sessions
- JWT
- OAuth
- CORS
- CSRF
- XSS
- Authentication
- Authorization

121. System Design Fundamentals

- Three-tier architecture
- Client-server architecture
- Monolithic applications
- Microservices
- REST API design
- Database design
- MongoDB schema design
- Caching
- Redis
- Load balancing
- Horizontal scaling
- Vertical scaling
- Message queues
- CDN concepts
- Authentication
- File storage
- High availability

Practice systems:

- URL shortener
- E-commerce application
- Chat application
- Job portal
- Learning Management System
- Notification service

122. Mock Interviews & Coding Practice

- JavaScript coding challenges
- TypeScript exercises
- Angular practical questions
- Node.js coding challenges
- MongoDB queries
- REST API design
- Debugging exercises
- Project explanation
- System design rounds

- Technical mock interviews
- HR mock interviews
- Resume review
- Interview communication

Recommended MEAN Stack Technology Stack

- Layer — Technologies
- Programming — JavaScript, TypeScript
- Web Fundamentals — HTML5, CSS3
- UI Styling — Bootstrap / Tailwind CSS
- Frontend — Angular
- Reactive Programming — RxJS
- Frontend State — Signals / Services / NgRx concepts
- Backend Runtime — Node.js
- Backend Framework — Express.js
- Database — MongoDB
- ODM — Mongoose
- APIs — REST, JSON
- Authentication — JWT, OAuth/OIDC concepts
- API Validation — Joi / Zod / express-validator
- API Documentation — Swagger / OpenAPI
- API Testing — Postman
- Real-Time — WebSockets / Socket.IO
- Caching — Redis
- Background Jobs — BullMQ concepts
- Testing — Jest, Supertest, Angular Testing
- Version Control — Git, GitHub
- Containers — Docker, Docker Compose
- CI/CD — GitHub Actions
- Cloud — AWS / Azure / GCP fundamentals
- Database Cloud — MongoDB Atlas
- Monitoring — Logging, health checks, observability fundamentals
- AI-Assisted Coding — GitHub Copilot, ChatGPT, Claude/Cursor
- Generative AI — LLM APIs, embeddings, introductory RAG
- Project Management — Agile, Scrum, Jira

Recommended Learning Sequence

Web Fundamentals → HTML → CSS → JavaScript → Advanced JavaScript → TypeScript → Git/GitHub → Angular → RxJS → Node.js → Express.js → MongoDB → Mongoose → REST API Development → Authentication & Security → Angular + Backend Integration → Testing → Redis → Real-Time Applications → Docker → CI/CD → Cloud Deployment → Microservices Fundamentals → AI-Assisted Development → Generative AI Integration → Capstone Project → Interview Preparation

Suggested Course Positioning

For Archer Infotech, I would position this course as:

AI-Assisted MEAN Stack Full-Stack Development Course

with the core stack:

MongoDB + Express.js + Angular + Node.js + TypeScript + REST APIs + Docker + Cloud + Generative AI

A strong differentiator would be to explicitly show students the complete application architecture:

Angular UI → Angular Services → Node.js/Express REST API → Mongoose → MongoDB

and then extend it to:

Angular → Node.js/Express → MongoDB + Redis + AI/LLM Services → Docker → Cloud

This makes the program more relevant than a traditional MEAN course because students learn **full-stack application development, API architecture, authentication, real-time systems, DevOps, cloud deployment, AI-assisted coding, and GenAI integration** as part of one coherent development path.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.